SUPERSONIC: Learning to Generate Source Code Optimizations in C/C++

Zimin Chen, Sen Fang, and Martin Monperrus

***Abstract*—Software optimization refines programs for resource efficiency while preserving functionality. Traditionally, it is a process done by developers and compilers. This paper introduces a third option, automated optimization at the source code level. We present SUPERSONIC, a neural approach targeting minor source code modifications for optimization. Using a seq2seq model, SUPERSONIC is trained on C/C++ program pairs (x_t, x_{t+1}), where x_{t+1} is an optimized version of x_t, and outputs a diff. SUPERSONIC's performance is benchmarked against OpenAI's GPT-3.5-Turbo and GPT-4 on competitive programming tasks. The experiments show that SUPERSONIC not only outperforms both models on the code optimization task but also minimizes the extent of the change with a model more than 600x smaller than GPT-3.5-Turbo and 3700x smaller than GPT-4.**

***Index Terms*—Code optimization, Seq2Seq learning, large language model.**

I. INTRODUCTION

SOFTWARE optimization refers to the process of refining a program so that it utilizes fewer resources, such as time, memory, CPU, and energy while preserving its original functionality. Traditionally, this task has been carried out by the developer and/or the compiler. The developer possesses deep human expertise to enhance the program by using a more efficient data structure or an algorithm with better complexity. On the other hand, the compiler can apply a range of automated optimizations on the intermediate representation that can significantly enhance the program's performance. Human developers optimize at the source code level, and compilers at the machine code level. In this work, we explore a third way: automated optimization at the source code level.

For developers, automatic source code optimization is invaluable. Firstly, it encompasses optimizations that are beyond the scope of compiler optimizations, high-level optimizations that a compiler cannot achieve with guarantees. For instance, refactoring an inefficient algorithm or modifying data structures to boost performance is beyond the compiler's scope. Secondly, it facilitates the optimization of legacy systems for which language and domain expertise have been lost over time, such as Fortran libraries or Cobol systems. Process-wise, automatic source code optimization can be used in modern code bases, with automated pull requests [1].

In this paper, we introduce SUPERSONIC, an innovative end-to-end system that employs a supervised machine-learning approach to the problem of automatic source code optimization. SUPERSONIC, implemented as a seq2seq model, learns the relationship between input programs and their optimized versions. Our process involves collecting a dataset of past source code optimizations, where each pair (x_t, x_{t+1}) consists of a base program x_t and its optimized counterpart x_{t+1}. Special attention has been given to creating such a high-quality training dataset. With this dataset, we then tailor a model and devise a data and training loop specifically for the optimization task.

At inference time, SUPERSONIC has three phases: canonicalization, diff-synthesis, and post-processing. During the canonicalization phase, SUPERSONIC removes source code comments and ensures a canonical code format (w.r.t. whitespaces, tabs, etc.) that has also been enforced at training time. The diff-synthesis phase takes the input program and predicts a diff-based output representation, which is similar to the typical Unix patch format. The post-processing phase is responsible for validating the predicted diff-based output representation and applying it to the original program. SUPERSONIC is implemented for C/C++ and trained on a large corpus of C/C++ program optimizations.

For the training and evaluation of SUPERSONIC, we use programs from three code competition websites. This is a perfect evaluation benchmark since problems on code competition websites are usually designed to have a large solution space and they also have a strong emphasis on optimization, running time and memory consumption. They are excellent for collecting training data as well as for assessing optimized programs. For evaluation, we compare against a strong baseline. SUPERSONIC competes with OpenAI's GPT-3.5-Turbo and GPT-4, two industrial large language models that have proven to be state-of-the-art in many software engineering tasks [2], [3], [4],

Manuscript received 29 September 2023; revised 23 June 2024; accepted 26 June 2024. Date of publication 22 July 2024; date of current version 14 November 2024. This work was supported in part by Wallenberg Artificial Intelligence, Autonomous Systems and Software Program (WASP) funded by Knut and Alice Wallenberg Foundation, and in part by the Swedish Foundation for Strategic Research (SSF). The computations were enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), in part by the Swedish Research Council under grant agreement 2022-06725. Recommended for acceptance by D. Lo. *(Corresponding author: Zimin Chen.)*

The authors are with KTH Royal Institute of Technology, 114 28 Stockholm, Sweden (e-mail: zimin@kth.se; sfang9@ncsu.edu; monperrus@kth.se).

Digital Object Identifier 10.1109/TSE.2024.3423769

© 2024 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

[5]. Our evaluation of 559 programs shows that SUPERSONIC outperforms both GPT-3.5-Turbo and GPT-4. Not only is the performance better on the task of optimizing C/C++ programs, but SUPERSONIC 's model is approximately 600x smaller[1] than GPT-3.5-Turbo and 3700x smaller[2] than GPT-4, making it more cost and energy-efficient.

SUPERSONIC is unique compared to the most related work, such as PIE4Perf [6] and DeepDev-PERF [7]. First, we take special care in removing essentially full re-implementations, and not optimized solutions from the dataset. Second, SUPERSONIC uses a novel diff-based output representation as opposed to generating the entire program, a feature that, as demonstrated in our evaluation, significantly boosts its effectiveness. Last, the evaluation of SUPERSONIC is done on third-party competition websites, giving our results strong external validity.

In summary, our contributions are:

- SUPERSONIC, a novel source code optimization technique based on state-of-the-art sequence-to-sequence learning. SUPERSONIC is able to generate source code level optimizations while retaining significant similarity with the original program.
- We show that SUPERSONIC outperforms GPT-3.5-Turbo and GPT-4 when submitting tentative optimizations to the official Codeforces website. It improves running time for 26.0% programs, compared to only 12.0% for GPT-3.5-Turbo and 4.0% for GPT-4.
- We investigate the optimization performance of SUPERSONIC, GPT-3.5-Turbo, and GPT-4 at various string similarity thresholds between the optimized and original program. We find that SUPERSONIC is better than GPT-3.5-Turbo and GPT-4 when the threshold is above 0.4. However, the threshold of 0.4 is close to a complete rewrite.
- We demonstrate that the diff-based output representation is better than outputting the full program. Our ablation study shows that changing the full program output representation to a diff-based output representation of SUPERSONIC boosts the optimization success rate by 2x at least.
- For the sake of open science and future research on this topic, we share all our code, data, and train models at https://github.com/ASSERT-KTH/Supersonic.

II. BACKGROUND

A. Large Language Models

A Large Language Model (LLM) is a deep learning (DL) model primarily used in natural language processing (NLP). LLMs are trained on large textual datasets to acquire a universal language representation, which can be refined for various tasks through supervised fine-tuning [8]. Most LLMs are based on the Transformer architecture [9] and can be categorized into three types: encoder-only [8], decoder-only [10], and encoder-decoder [11].

[1]We assume it is the same size as GPT3, which is a 175B parameter model.

[2]There are no official numbers on the number of parameters of GPT-4, we assume it is a trillion parameter model.

Encoder-only LLMs, like BERT [8], use multiple Transformer encoder layers and are trained with two objectives: masked language modeling and next sentence prediction. Decoder-only LLMs, such as the GPT family [10], [12], use the Transformer decoder and are pre-trained with a next token prediction objective. Encoder-decoder LLMs, like T5 [13], combine both encoder and decoder layers and are pre-trained with a fill-in-the-blank objective. Literature [14] and [15] provide a detailed introduction of LLMs.

LLMs are increasingly utilized in the field of software engineering (SE) to expedite automation processes. An example of this is CodeBERT [16], an LLM designed specifically for programming language. This model was initially trained on a broad, cross-programming language (PL) corpus. Subsequently, supervised fine-tuning allows its application to various SE tasks, such as code searching [17] and summarization [18]. To augment the efficacy of LLMs in handling code, Wang et al. introduced CodeT5 [19], which was pre-trained using an innovative method known as identifier-aware denoising training. Additional training objectives include identifier tagging, masked identifier prediction, and bimodal dual generation. All of these objectives facilitated the model's ability to effectively comprehend both programming and natural language. Fang et al. proposed RTA [20], a domain-specific LLM trained on bug reports that can be used for various bug report related tasks by fine-tuning.

B. The State-of-the-Art Models at OpenAI

Recently, OpenAI released GPT-3.5-Turbo [21] and GPT-4 [22], two groundbreaking NLP systems specializing in generating text that closely mirrors human language. These state-of-the-art systems owe their performance to the Transformer decoder-based architecture, known as GPT [10]. GPT-3.5-Turbo's training involves a two-stage process: unsupervised pre-training and subsequent instruction fine-tuning. In the pre-training phase, the model learns from an extensive corpus of Internet data dated until September 2021. It is important to underscore that the system does not possess specific knowledge about the documents that comprise its training data and cannot directly access any database or document. The second stage, instruction fine-tuning, involves training the model on a smaller dataset with human feedback, utilizing a technology called reinforcement learning from human feedback [23]. With suitable instructions, also called prompts, GPT-3.5-Turbo can serve a wide array of applications, including but not limited to generating code [3], debugging [24], acting as a tutoring assistant [2], translating languages [25], and even simulating characters for video games [26]. Following the success of GPT-3.5-Turbo, OpenAI introduced GPT-4, an even more advanced model with an increased number of parameters and refined training techniques. The GPT-4 is better than GPT-3.5-Turbo on understanding and generating text, improving its predecessor in terms of context accuracy, diversity of output, and adaptability across diverse applications [22]. In this work, we explore the capability of GPT-3.5-Turbo and GPT-4 in code optimization.

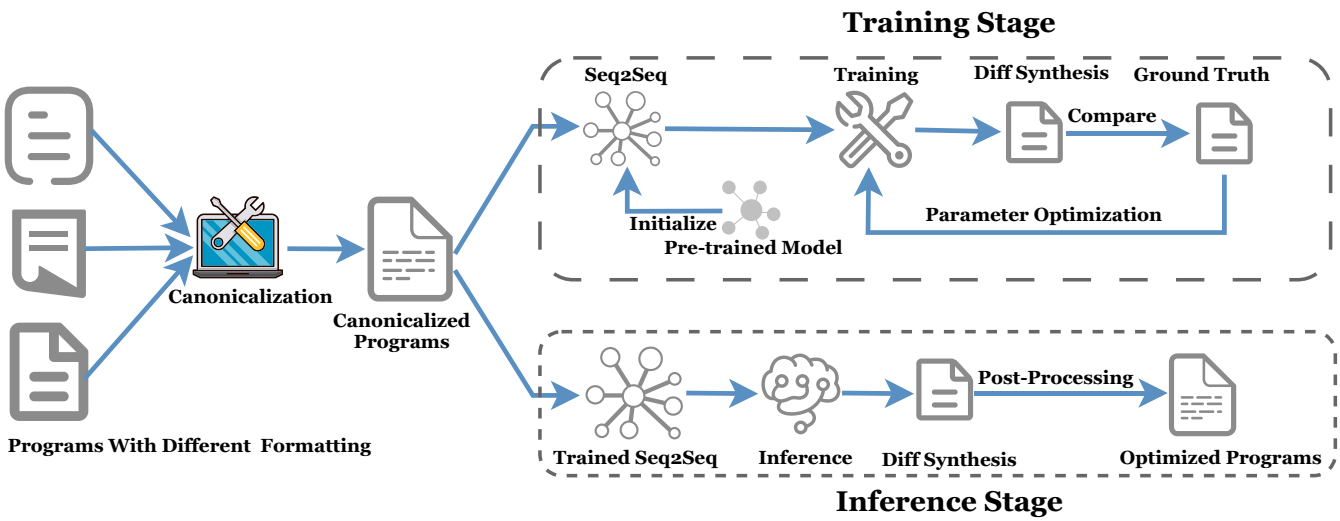


Fig. 1. The pipeline of SUPERSONIC for code optimization.

C. Sequence-to-Sequence Learning

Sequence-to-Sequence (Seq2Seq) learning [27], a concept introduced by the NLP community, is a machine learning paradigm that models conditional probability distributions for sequence outputs based on sequence inputs. This is usually implemented through an end-to-end neural network structure: an encoder and a decoder. Specifically, the encoder maps the input sequence into a fixed-length vector. To illustrate, taking code optimization as an example, the encoder could process a section of code requiring optimization, such as a C++ program, and convert it into a compact vector representation. Leveraging the powerful learning capabilities of neural networks, this vector can effectively capture the semantic information of the input sequence. Subsequently, the decoder utilizes this vector to produce the output sequence. Continuing with our example, the decoder could yield a C++ program optimized for some objectives, e.g., execution speed or memory usage. Training Seq2Seq models are typically conducted end-to-end, tuning the parameters to maximize the likelihood of the correct output sequence given the input sequence. Thus, Seq2Seq learning is a robust framework for tackling generation tasks involving intricate, variable-length sequence transformations.

III. TECHNICAL SOLUTION: SUPERSONIC

A. Overview

SUPERSONIC leverages seq2seq learning initialized with pre-trained models to generate optimized C/C++ programs. Importantly, the optimized programs are syntactically similar to the original program, meaning that the optimization only changes a few lines of the program. It is done by training SUPERSONIC on program pairs, (x_t, x_{t+1}), where x_t and x_{t+1} are functionally equivalent, with a small edit distance. SUPERSONIC consists of three phases: 1) Canonicalization, 2) Diff synthesis, 3) Post-processing.

Fig. 1 shows the training and inference pipeline of SUPERSONIC for code optimization from canonicalization to post-processing. The canonicalization phase canonicalizes the program coding style (whitespaces, tabs, and newlines) and removes comments before it is used as the input to the diff synthesis phase. The diff synthesis phase generates multiple optimization candidates in a diff-based output representation. The post-processing phase checks for the well-formedness of the diff-based output representation, and then applies the output as a patch on the original program. For the training stage of SUPERSONIC, the file is first canonicalized, then it is used as input to the seq2seq model. The seq2seq model generates a diff-based representation in the diff synthesis phase, and it is compared with the ground truth to update the parameter to the seq2seq model in an iterative loop. For the inference stage of SUPERSONIC, the input still goes through the canonicalization and diff synthesis phase, but since there is no ground truth, the output is applied to the original program in the post-processing phase.

In the following sections, we illustrate each phase of SUPERSONIC in detail and describe our training dataset.

B. Canonicalize Training Source Code

The goal of the canonicalization phase is to canonicalize program pairs (x_t, x_{t+1}) before they are used as input to the machine learning model. In this phase, we first remove source code comments and apply a unique, consistent code style.

We remove source code comments because they are subjective in nature, varying greatly in style, quality, and content. Meanwhile, although they do not affect the logical flow of the code, including comments makes the input program longer, which impacts negatively on the inference performance of the language model [28].

Input programs also vary in coding styles, for example number of whitespace, newlines, and how brackets are placed. These are all tokenized and used as input to the machine-learning model. Therefore we use a single coding style to let the machine learning model focus on the functionality of the program.

```
#include <algorithm>
#include <iostream>
#include <map>
using namespace std;
using ll = long long;
int main() {
  ll N, ans{};
  cin >> N;
  map<string, int> m;
  string S;
  while (cin >> S) {
    sort(begin(S), end(S));
    ans += m[S]++;
  }
  cout << ans << endl;
}
```

Listing 1: A user submission to the Green Bin problem at AtCoder

```
#include <algorithm>
#include <iostream>
#include <unordered_map>
using namespace std;
using ll = long long;
int main() {
  ll N, ans{};
  cin >> N;
  unordered_map<string, int> m;
  string S;
  while (cin >> S) {
    sort(begin(S), end(S));
    ans += m[S]++;
  }
  cout << ans << endl;
}
```

Listing 2: The improved solution by the same user. *unordered_map* that do not sort the mapping is used instead. The changed lines are highlighted in red.

```
 #include <iostream>
−#include <map>
+#include <unordered_map>
 using namespace std;

   cin >> N;
−  map<string, int> m;
+  unordered_map<string, int> m;
   string S;
```

Listing 3: The diff-based output representation by SUPERSONIC, the '-' and '+' are part of the output of the network.

Fig. 2. The comparison between the full program and the diff-based output representation generated by SUPERSONIC. When tokenized, the fully optimized program contains 124 tokens whereas the diff-based output representation contains 71 tokens. The difference will be even bigger if the original solution is longer. A longer output increases the probability of generating an erroneous program because of accumulated mistakes.

We use *GCC* (using the command "*gcc -fpreprocessed -dD -E -P*") to remove source code comments and *clang-format* (using the command "*clang-format –style=llvm*") to format all C/C++ programs according to LLVM coding style.

C. Output Representation & Diff Synthesis

A key aspect of SUPERSONIC is that it generates a diff-based output representation instead of generating the whole optimized program. The diff-based output representation is the Unix unified diff format between x_t and x_{t+1} with one line context, i.e. one line before and after the change. Compared to the original diff format, the diff header that specifies the filenames and changed line numbers is removed. Multiple change hunks are concatenated with newlines to represent a change that modifies multiple locations. In contrast, most related works [6], [7] employ a full program output representation. We use the diff-based output representation because generating a longer output sequence increases the probability of errors due to accumulated mistakes. It is widely known that seq2seq models performance decreases with the output length [29].

Fig. 2 shows an example of the diff-based output representation generated by SUPERSONIC. This example is a user submission to AtCoder - Green Bin problem[3]. The task is to find the number of anagrams from a list of strings. The original solution, Listing 1 uses a sorted mapping *map* to store pairs of strings and integers. The improved solution, Listing 2, by the same user used *unordered_map* that does not sort the mapping compared to *map*. This change improved the running time from 144 ms to 97 ms, and memory consumption from 11136 B to 10660 B. SUPERSONIC 's diff-based output representation of Listing 2 is shown in Listing 3. The diff-based output representation contains 71 tokens instead of 124 tokens when tokenizing the full improved program. As we will see in the evaluation, the shorter token length helps the model to capture knowledge with fewer tokens.

[3]https://atcoder.jp/contests/abc137/tasks/abc137_c

D. Post-Processing Synthesized Code

During the inference stage of SUPERSONIC, the diff-based output representation from the diff synthesis phase needs to be patched on the original program to get the full source code of the predicted optimized program. We greedily match each predicted changed hunk to the original program using the one-line context, and then apply the changed hunk. If we fail to match the context lines in the original program, it means that the diff synthesis phase has generated a malformed diff-based output representation and we simply discard it.

E. Training Loop

The core machine learning model of SUPERSONIC that generates and predicts the diff-based output representation is implemented as a transformer-style seq2seq model. The encoder and decoder are initialized with CodeBERT which is pre-trained on C++ code, CodeBERT-CPP [30]. CodeBERT is a pre-trained LLM on source code and natural language [16]. The training objective of CodeBERT is masked language modeling, where the model predicts the token masked by a special mask token, and replaced token detection, where the model predicts if a token is the original one or not. CodeBERT trained on these two objectives achieves a robust understanding of code semantics and syntax. By initializing our model with weights from CodeBERT, we leverage this rich foundation of code semantics and structure, which accelerates convergence during training and can lead to better overall performance [31].

TABLE I
OUR DATASET STATISTICS. THE VALUES IN PARENTHESES OF THE x_t LOC, x_{t+1} LOC AND SUPERSONIC OUTPUT LINES COLUMNS ARE THE LOWER AND UPPER QUARTILE

Split	Size	x_t LOC	x_{t+1} LOC	SUPERSONIC output lines	Codeforces solutions	AtCoder solutions	AIZU solutions
Train	312876	(35, 91)	(35, 90)	(5, 16)	276714	28665	7497
Validation	1000	(36, 88)	(36, 88)	(5, 15)	877	96	27
Test	559	(33, 86)	(33, 86)	(5, 17)	300	-	259
Total	314435	(35, 91)	(35, 90)	(5, 16)	277891	28761	7783

TABLE II
COLLECTED ATTRIBUTES FOR EACH SUBMISSION TO AIZU, ATCODER, AND CODEFORCES

Attribute	Description
origin	The origin of the submission: AIZU, AtCoder, or Codeforces
author	The author of the submission
contest_id	ID of the contest/problem
submission_id	ID of the submission
creation_time	When the submission is created
problem	The problem name
programming_language	The programming language: C or C++
cpu_time	The execution time
memory	The memory consumption
source_code	The submission source code

Our seq2seq model is trained to generate x_{t+1} by using x_t as the input of the program pair (x_t, x_{t+1}) of the training set. We reuse the tokenizer CodeBERT-CPP, which is based on the subword tokenization from WordPiece with 50 265 as the vocabulary size [32]. The total amount of parameters for the seq2seq model is 278M.

F. Training Dataset

Training SUPERSONIC to optimize C/C++ programs requires a dataset of source code optimization program pairs (x_t, x_{t+1}). In our work, we collect source code optimization program pairs from code competition websites. Code competition websites usually record the execution time and memory usage of all submitted solutions to each problem. It makes them the ideal source to crawl high-quality and accurate source code optimization program pairs.

In our work, we use data from Codeforces[4], as well as AIZU[5] and AtCoder[6] originally from the CodeNet dataset [33] to build our goal dataset. For the Codeforces dataset, we first use the Codeforces API to get all the past contests and submissions to each contest. However, the Codeforces API does not return the source code, therefore we use another source[7] to find the source code for each submission. For submissions to AIZU and AtCoder, the CodeNet dataset already provides all the necessary information that we need to process the data. All the submissions that we collect are accepted by each competition website, therefore their correctness is guaranteed. The attributes that we collect from each competition website are shown in Table II.

To ensure the relevance of our dataset, we specifically focused on program pairs (x_t, x_{t+1}) from the same author, where x_{t+1} strictly improves either on the running time or memory usage over x_t. By requiring the submissions to be in chronological order, we aimed to capture the author's progress and improvements over time. This approach allows us to observe the evolution of optimizations employed by an author and to analyze the specific code changes that led to enhanced performance. Collecting these pairs of programs directly from the same author, in chronological order, minimizes confounding factors such as coding style, enabling us to attribute improved performance to code changes. This careful data collection guarantees monotonic improvements.

The primary objective of SUPERSONIC is to generate optimized programs while minimizing the extent of changes from the original code. Intuitively, we want to train the system to generate optimizations that modify a few lines of code only. To do this, we filter all pairs of submissions based on the following criteria:

1) At most 20 lines are changed between x_t and x_{t+1}.
2) At most 20% of lines are changed between x_t and x_{t+1}.
3) At least a string similarity score of 0.8 between x_t and x_{t+1}.

The string similarity metric provides a value between 0.0 and 1.0, with 0.0 indicating two strings are entirely dissimilar and 1.0 indicating a perfect match between the strings. The string similarity metric we use is the *SequenceMatcher* from the Python *difflib* library. It uses a variant of the Gestalt pattern matching string-matching algorithm [34].

In total, we collected 20M Codeforces submissions and 6M submissions from the CodeNet dataset. Then, we extracted 746K and 138K submission pairs from Codeforces and CodenNet which either improves on the running time or memory consumption. After filtering the submission pairs, we split the remaining submission pairs into train, validation, and test sets, consisting of 312 876, 1000, and 559 samples, respectively. The pre-training dataset used to pre-train CodeBERT is the C++ source code of the GitHub-Code-Clean dataset under the CodeParrot project on Huggingface[8]. The full dataset statistics are shown in Table I. From the table, we can observe that AtCoder and AIZU datasets are relatively small, which may entail an overfitting risk if we train the model on a single dataset. To resolve this issue, we combine samples from the three code competition platforms to build a large training dataset, which

[4]https://codeforces.com
[5]https://judge.u-aizu.ac.jp
[6]https://atcoder.jp
[7]https://codeforces.com/blog/entry/94755
[8]https://huggingface.co/datasets/codeparrot/github-code-clean

helps effectively alleviate the overfitting risk in the training phase, boosting the generalization ability of the model.

IV. Experimental Setup

In this section, we describe our experimental setup to evaluate SUPERSONIC against the state-of-the-art related work.

A. Research Questions

RQ1 - Effectiveness How does SUPERSONIC perform on the task of code optimization while minimizing the extent of changes?

RQ2 - Advanced Prompting ChatGPT How does SUPERSONIC compare against advanced prompting of ChatGPT?

RQ3 - Optimization Focus How is SUPERSONIC effective at minimizing the amount of changes?

RQ4 - Ablation Study To what extent is the diff-based output representation better than the full program output representation?

These research questions are framed to address different aspects of the problem domain. RQ1 compares SUPERSONIC against GPT-3.5-Turbo and GPT-4 on running time and memory consumption optimizations, while minimizing the extent of changes, by submitting predicted program to code competition websites. RQ2 further compares SUPERSONIC against GPT-3.5-Turbo and GPT-4 with advanced prompting, incl. Chain-Of-Thought [35], under the same goal as RQ1. RQ3 in contrast to RQ1 and RQ2, relaxes the constraint of minimizing the extent of changes. Lastly, RQ4 explores the impact of using our diff-based output representation compared to using the full program, to validate our core design.

B. Test Dataset

Our test dataset, described in subsection III-F, consists of user submissions of code competition problems. The problems are of different difficulties and are designed with possible optimization in mind. These problems are known to be hard for language models. For example, GPT-4 is ranked at the bottom 5% when trying to Codeforces problems [22]. The code competition website is also used to compute and report the running time and memory usage.

Our full test dataset consists of 559 user submissions (see Table I), of which 300 are Codeforces user submissions, and 259 are AIZU user submissions. In the test dataset, we excluded the submissions from the AtCoder dataset because we found that the evaluation results from the AtCoder platform are unreliable due to inconsistencies with problem ID matching. The 559 user submissions, along with their predictions, and the collected running time and memory consumption are used to answer all research questions. Our test set does not overlap with any examples in the pre-training or our own training dataset. The check is done by checking all pairs of examples in the test set and the pre-training dataset for exact matches.

C. Methodology for RQ1

The goal of RQ1 is to study whether SUPERSONIC, OpenAI's GPT-3.5-Turbo and GPT-4 can optimize programs. Recall that code optimization typically involves making edits to only a portion of the original program, without completely changing it: Alali, Kagdi, and Maletic found that the majority of changes are classified as small, changing 6-46 lines [36]. With that respect, the experiment takes care of discarding re-implementations of the original program. Smaller edits are also less likely to contain bugs [37] and more likely to be accepted as a pull request [38].

Concretely, SUPERSONIC, GPT-3.5-Turbo and GPT-4 are asked to generate 10 optimization predictions per original program in our test set (subsection III-F). Afterward, we submit these predictions to the corresponding code competition website to evaluate their correctness and efficiency. If a prediction can pass all test cases, we regard it as a correct solution. Then, in order to discard re-implementations, we filter those where the string similarity value compared to the original program is lower than 0.8, the same value we used in subsection III-F. Next, the metrics that we use to compare SUPERSONIC and GPT-3.5-Turbo are running time and memory usage (from [6]):

- **Percent optimized** - %OPT: The percentage of optimized programs in the test set.
- **Performance improvement** - PI: The average improvement in running time or memory usage of the best-optimized program among the predictions. If *old* and *new* are running time or memory usage of the original and optimized program, then $\text{PI} = \frac{old}{new}$. We report the average PI over the test set.

We compute these metrics for strictly better memory improvements and for strictly better running time improvements. We realize that code competition measurements are not perfectly precise and tend to have some variation. To account for this noise, we only consider a program to be truly more optimized if the performance improvement is at least 20%, i.e. PI is at least 1.2. We calculate the highest improvement in terms of the above two metrics and regard the prediction on these two metrics as the indicator of the best-optimized program. Note that evaluating all predictions is computationally expensive and time-consuming. By focusing on the top-10 predictions, we have a good trade-off between exploration and cost. With this setup, we use the search power of the beam, while still being able to capture the best-optimized program in a sampling of 10. Given our replication package, future researchers will be able to study the impact of different values of beam search.

We utilize OpenAI's official APIs to call GPT-3.5-Turbo and GPT-4 for generating predictions. As per OpenAI's official API documentation[9], the APIs are updated to incorporate the latest model iteration approximately two weeks subsequent to its release[10]. To ensure clarity regarding the API version employed in our experiments, our predictions spanned from April 17 to July 5 2023, whereas for GPT-4, predictions were conducted between August 21 and August 25 2023. The GPT-3.5-Turbo and GPT-4 prompt we use to generate the predictions is shown

[9]https://platform.openai.com/docs/models/gpt-3-5

[10]https://help.openai.com/en/articles/6825453-chatgpt-release-notes

system prompt: I want you to act as an experienced C and C ++ developer and your task is to optimize my written C or C ++ programs. I want you to optimize my program for running time and memory usage. I will type my C or C++ program and you will optimize the program and return the optimized program. I want you to only reply with the fixed program inside one unique code block, and nothing else. Do not write explanations.

user prompt: [CODE]

Listing 4. The prompt used for querying OpenAI models to generate more optimized programs.

in Listing 4. We use a carefully designed prompt per the best practices[11]. The prompt instructs the model to act as an experienced C/C++ developer and optimize the given program with respect to running time and memory usage. We do not instruct GPT-3.5-Turbo and GPT-4 to generate unified diff since these models have seen few diffs at training time and have limited diff capability [39], making it hard to use them to generate meaningful and effective diff-based outputs.

D. Methodology for RQ2

The goal of RQ2 is to compare SUPERSONIC against advanced prompting of OpenAI's GPT-3.5-Turbo and GPT-4. Similar to RQ1, we utilize OpenAI's official APIs to call GPT-3.5-Turbo and GPT-4 for generating predictions. To ensure clarity regarding the API version employed in our experiments, our experiments spanned from January 22, 2024, to February 27, 2024. The prompt we use to generate the predictions is shown in Listing 5, which is designed by combining the prompt techniques from recent work [35], [40], [41]. The prompt instructs the model to act as an experienced C/C++ developer and optimize the given program step by step, with respect to running time and memory usage. Then, we have three specific instructions, different from the baseline prompt discussed in RQ1. [INSTRUCTION 1] instructs the model to optimize the given program by only making small modifications. [INSTRUCTION 2] further requires the model not to rewrite the full program. Finally, [INSTRUCTION 3] prompts the model to reason step by step, in a chain-of-thought way, where the given natural language explanations for each step are meant to help the model to produce better patches.

E. Methodology for RQ3

In RQ3, the goal is to study to what extent SUPERSONIC, GPT-3.5-Turbo and GPT-4 optimize by re-implementing. To do that, we compare SUPERSONIC, GPT-3.5-Turbo, and GPT-4 at various string similarity thresholds using the same protocol as RQ1. Also, we study the string similarity distribution for all predictions to see how much the programs are changed for SUPERSONIC and GPT-3.5-Turbo. Then, we plot the %OPT value for different string similarity thresholds between 0 and 1.

[11]https://github.com/f/awesome-chatgpt-prompts

I want you to act as an experienced C/C++ developer and your task is to optimize the given C/C++ program below.

I want you to optimize the program for running time and memory usage.

I will type the C/C++ program, you will return the optimized program.
[INSTRUCTION 1] The optimized program should make a small modification to the original one.
[INSTRUCTION 2] Do not completely rewrite the program.

[INSTRUCTION 3] Please explain the optimization step by step first and then generate the optimized program in a code block.

[YOUR CODE]

Listing 5. The advanced prompt used for querying OpenAI models to generate optimized programs. ([INSTRUCTION x] is not actually part of the prompt, shown for explanatory purposes.)

F. Methodology for RQ4

The goal of RQ4 is to evaluate if the diff-based output representation is a better code representation than the full program representation. We use the same model architecture and the same dataset split described in subsection III-F and used in RQ1, RQ2, and RQ3. The only difference is that one model learns to generate the full program, while the other learns to generate the diff-based output representation described in subsection III-C. Similar to RQ1 described in subsection IV-C, 10 predictions are generated per sample in the test set and predictions with a string similarity value below 0.8 are filtered. The rest are submitted to the corresponding code competition website. We compute %OPT and SP on running time and memory consumption.

V. RESULTS

A. Answer to RQ1 (Effectiveness)

Table III shows the comparison between SUPERSONIC, GPT-3.5-Turbo, and GPT-4 on generating source code optimizations for Codeforces and AIZU submissions. For the Codeforces submissions, SUPERSONIC generates optimizations that improve 26.0% (78) and 8.0% (24) of the programs on running time and memory consumption respectively. This shows the advantage and efficacy of SUPERSONIC over both GPT-3.5-Turbo and GPT-4, illustrating its capability for optimizing a range of problems, even when compared against the most advanced generative models currently available. This is despite the fact that SUPERSONIC is 600x and 3700x smaller than GPT-3.5-Turbo and GPT-4. For AIZU submissions, both SUPERSONIC and GPT-3.5-Turbo perform at a similar level for running time and memory optimization, while GPT-4 is still performing worse. The reason why SUPERSONIC has a lower performance for AIZU submissions than Codeforces submissions might be because

TABLE III
PERFORMANCE OF TWO PROMPTS FOR GPT-3.5-TURBO AND GPT-4, COMPARED TO SUPERSONIC (300 CODEFORCES PROGRAMS AND 259 AIZU PROGRAMS)

Metrics		GPT-3.5-Turbo (basic)		GPT-4 (basic)		GPT-3.5-Turbo (advanced)		GPT-4 (advanced)		SUPERSONIC	
		Running time	Memory	Running time	Memory	Running time	Memory	Running time	Memory	Running time	Memory
Codeforces	%OPT	12.0% (36)	3.7% (11)	4.0% (12)	1.3% (4)	0.7% (2)	0% (0)	7.7% (23)	5.3% (16)	26.0% (78)	8.0% (24)
	PI	3.80×	11.57×	2.73×	1.53×	2.86×	0×	4.46×	83.07×	2.65×	1.81×
AIZU	%OPT	4.6% (12)	1.9% (5)	1.5% (4)	0.8% (2)	0.8% (2)	0.8% (2)	3.1% (8)	0.8% (2)	3.5% (9)	1.2% (3)
	PI	6.87×	3.69×	4.88×	2.45×	3.11×	1.43×	3.17×	3.79×	2.82×	1.23×

```
@@ −28,12 +28,7 @@
    sort(v.rbegin(), v.rend());
    for (int i = 0; i < v.size(); i++) {
      sm += v[i];
−     if (i == 0)
−       arr[0] = v[0];
−     else {
−       arr[i] = sm / (i + 1);
−     }
−     if (arr[i] >= m)
+     if (sm / (i + 1) >= m)
        cnt++;
    }
    cout << cnt << E;
 #include <algorithm>
 #include <cmath>
```

Listing 6. Unified diff for one optimization by SUPERSONIC. It simplifies if statements, and removes the need of array *arr*. The string similarity between the original and predicted program is 0.95. The original problem is the *Middle Class* problem on Codeforces[12].

```
@@ −1,3 +1,6 @@
+#pragma GCC optimize("Ofast")
+#pragma GCC optimize("unroll−loops")
+#pragma GCC target("avx,avx2,sse,sse2")
 #define _CRT_SECURE_NO_WARNINGS
 #include <algorithm>
 #include <cmath>
```

Listing 7. Unified diff for another optimization by SUPERSONIC. It add several GCC pragmas that instructs the GCC compiler on how to handle the code. These optimize the code's execution speed and enable specific compiler optimizations and instruction sets. The string similarity between the original and predicted program is 0.99. The original problem is the *WW* problem on AIZU[13].

our training data contains more than 80% program pairs from Codeforces, hence have better learned on that type of problem. The problemson Codeforces and AIZU are indeed different, Codeforces being competitive in nature with regular contests dividing participants with ratings, AIZU being primarily an educational platform.

Listing 6 and Listing 7 are two examples of optimization by SUPERSONIC, as a diff between the original and generated program. The first example simplifies the if statements to a single if statement and removes the need for the array *arr* that keeps the intermediate values. The generated example reduces the memory consumption from 2.64MB to 1.56MB, saving 41% of memory usage. The second example adds three GCC pragma directives that are instructions to the GCC compiler on how to compile the code. *optimize("Ofast")* enables all *-O3* optimizations along with those applicable to all programs compliant with the standard. *optimize("unroll-loops")* unroll loops, a technique that attempts to optimize the running time by sacrificing the binary size. *target("avx,avx2,sse,sse2")* enables the use of specific instruction set extensions for vector operations. The generated program reduces the running time to 7ms from the original 17ms, which is a significant 60% decrease. This also demonstrates that SUPERSONIC could learn some effective compilation optimization patterns, in addition to optimization by changing the data structure or algorithm.

We give a breakdown of all 5590 predicted programs (10x559 predictions, for every program in the test set), for SUPERSONIC. The post-processing phase of SUPERSONIC removed 2045 programs because of malformed diff-based output representation that can not be applied to the original program. For the remaining 3545 programs that we submitted for evaluation, 1686 programs are accepted as valid solutions, they compile and pass the functional tests, showing the capabilities of SUPERSONIC to output proper syntax and to maintain semantics. Among the 1686 accepted programs, 356 programs either improved the running time or memory consumption.

RQ1: SUPERSONIC, a relatively small neural network, is able to automatically generate source-code level optimized programs. It is capable of producing, short, targeted optimizations, including appropriate compiler directive addition.

B. Answer to RQ2 (Advanced Prompting ChatGPT)

Table III shows the comparison between SUPERSONIC and advanced prompting OpenAI's GPT on generating source code optimizations for Codeforces and AIZU submissions. For the Codeforces submissions, SUPERSONIC generates optimizations that improve 26.0% (78) and 8.0% (24) of the programs on running time and memory consumption, respectively. This is higher than GPT-3.5-Turbo and GPT-4 for both prompts. For AIZU submissions, both SUPERSONIC and GPT-4 perform at a similar level for running time and memory optimization, but GPT-4 is still performing worse.

To provide a more detailed analysis, we examine the overlap in the optimized submissions among GPT-3.5-Turbo, GPT-4, and SUPERSONIC. For Codeforces submissions, half of the programs optimized by GPT-3.5-Turbo can also be optimized by SUPERSONIC in terms of running time. More

[12]https://codeforces.com/problemset/problem/1334/B
[13]https://onlinejudge.u-aizu.ac.jp/problems/1537

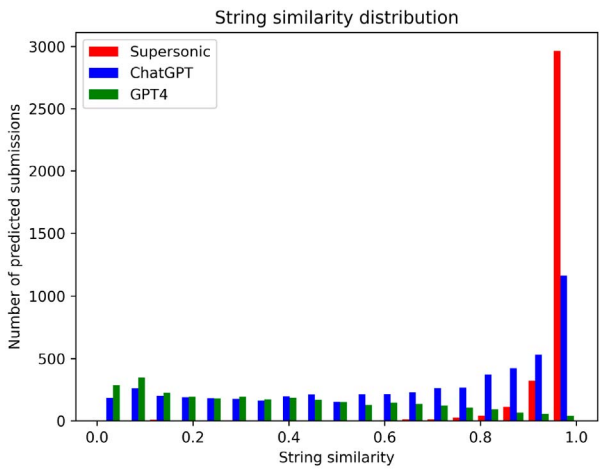


Fig. 3. The string similarity distribution for the similarity between the predicted submission and the original solution. Best viewed in color.

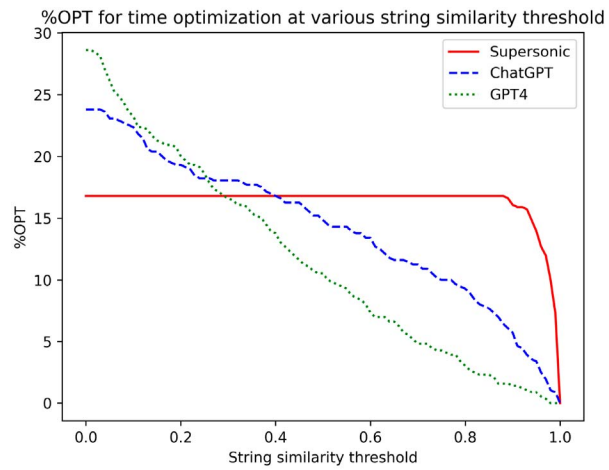


Fig. 4. %OPT for running time optimization of SUPERSONIC, GPT-3.5-Turbo and GPT-4 if we limit the string similarity at various thresholds between the predicted submission and the original solution.

specifically, for programs optimized by these two models uniquely, we observe that GPT-3.5-Turbo focuses on the optimization of I/O operations and data structures. Except for these two types of optimization, in addition, SUPERSONIC also pays attention to compilation optimization. Note this is only an observational correlation, and future extensive experiments are needed to establish causation. In the general case, it is hard to identify and summarize the general code features that are optimized by different models.

Overall, the findings of this new experiment fully confirm those of RQ1. Interestingly, we observe that GPT-4 outperforms GPT-3.5-Turbo when utilizing advanced prompts for both datasets, which suggests that GPT-4 possesses a superior ability to exploit advanced prompting and to adhere to high-level instructions.

RQ2: SUPERSONIC is able to better optimized programs than OpenAI models, incl. GPT-3.5-Turbo and GPT-4. This holds even for advanced prompting of GPT-4. GPT-4 is clearly better at handling complex optimization instructions.

C. Answer to RQ3 (Optimization Focus)

The string similarity distribution for all generated programs is shown in Fig. 3. GPT-3.5-Turbo and GPT-4 generate prediction at all string similarity levels, which means they sometimes generate target optimizations and sometimes rewrite the whole program. On the contrary, most of SUPERSONIC 's predictions have a high string similarity (red distribution shifted to the right), which shows SUPERSONIC generates much more precise edits to the programs under optimization. In addition, we see that most generated programs have a similarity higher than 0.8, which is very consistent with the training data curation, as described in subsection III-B, showing the soundness of the end-to-end training and inference loop.

Fig. 4 shows how the %OPT metric changes with different string similarity thresholds for generated programs that optimize the running time. The y-axis is the %OPT value. The x-axis is the string similarity threshold, meaning that we filter all generated programs with a lower string similarity score than the corresponding x value. For $x = 0$, it means that we include all generated programs, and for $x = 1$, all generated programs are filtered out. Reading the graph from right to left, we can see that SUPERSONIC is the best (red curve at the top) until the string similarity threshold is close to 0.4. This means the string similarity threshold of 0.8 in RQ1 was not a magic, unstable one. Rather, it can be changed to a large degree and SUPERSONIC would still be the best performing tool. From the same graph, we can also see that GPT-4 performs better than GPT-3.5-Turbo when the threshold is 0, but the relationship reverses when the threshold is higher. Overall, it is clear that GPT-4 is generating more full rewrites than GPT-3.5-Turbo.

To give an idea of how dissimilar two programs are when the similarity score is low, Listing 8 shows a GPT-3.5-Turbo generated program with similarity 0.36. It is clear that most lines are changed, including variable names. Other significant rewrites by GPT-3.5-Turbo in that example include *std::transform* substituted with for-loops and *char* type used instead of integer for comparison.

RQ3: SUPERSONIC is the best-performing model compared to GPT-3.5-Turbo and GPT-4 even when we lower the similarity threshold. GPT-3.5-Turbo and GPT-4 tend to optimize by fully rewriting the program which is argued to be a different task than optimization, closer to re-implementation. To the best of our knowledge, SUPERSONIC is the best model for generating precise, targeted, and effective source code optimizations.

D. Answer to RQ4 (Ablation Study)

For this research question, we focus on investigating the efficacy of two distinct code output representations when training

```
@@ −1,18 +1,24 @@
−#include <bits/stdc++.h>
+#include <iostream>
+#include <cstring>
 using namespace std;
+
 int main() {
−  string s;
−  cin >> s;
−  short i, t = 0;
−  for (i = 0; i < s.size(); i++) {
−    if (s[i] > 96)
−      t++;
−  }
−  i = s.size() − t;
−  if (t < i)
−    transform(s.begin(), s.end(), s.begin(), ::toupper);
−  else
−    transform(s.begin(), s.end(), s.begin(), ::tolower);
−  cout << s;
−  return 0;
−}
+    char s[1000001];
+    cin >> s;
+    int upper = 0, lower = 0;
+    for (int i = 0; i < strlen(s); i++) {
+        if (s[i] >= 'a' && s[i] <= 'z') lower++;
+        else upper++;
+    }
+    if (upper > lower) {
+        for (int i = 0; i < strlen(s); i++) {
+            s[i] = toupper(s[i]);
+        }
+    } else {
+        for (int i = 0; i < strlen(s); i++) {
+            s[i] = tolower(s[i]);
+        }
+    }
+    cout << s;
+    return 0;
+}
```

Listing 8. Unified diff for one example prediction from GPT-3.5-Turbo. Almost all source code lines are changed and the string similarity between the prediction and original program is 0.36. This cannot be considered as a targeted optimization. The original problem is the *Word* problem on Codeforces[14].

TABLE IV
SUPERSONIC WITH DIFF-BASED OUTPUT REPRESENTATION (SUPERSONIC DIFF) AND FULL PROGRAM (SUPERSONIC FULL) AS OUTPUT. WITHIN THE PARENTHESES FOR %OPT IS THE ABSOLUTE NUMBER OF OPTIMIZED PROGRAMS

Metrics		SUPERSONIC Full		SUPERSONIC Diff	
		Running time	Memory	Running time	Memory
Codeforces	%OPT	11.0% (33)	2.7% (8)	26.0% (78)	8.0% (24)
	PI	3.28×	1.57×	2.65×	1.81×
AIZU	%OPT	1.5% (4)	0.0% (0)	3.5% (9)	1.2% (3)
	PI	3.24×	0.00×	2.82×	1.23×

our SUPERSONIC model. One version of SUPERSONIC is trained to generate an entire program as output, exemplified in Listing 2. The second version is trained to produce a diff-based output representation, which can be seen in Listing 3. Table IV shows the comparison between the two versions on the predicted submission to the Codeforces and AIZU code competition website.

[14]https://codeforces.com/problemset/problem/59/A

SUPERSONIC with a diff-based output representation is clearly better than SUPERSONIC with a full program output. The diff-based output representation optimizes more programs on both Codeforces (26.0% versus 11.0% for running time, 8.0% versus 2.7% for memory consumption) and AIZU (3.5% versus 1.5% for running time, 1.2% versus 0.0% for memory consumption). Considering the presented data, it is evident that the diff-based output representation of SUPERSONIC consistently outperforms its full program counterpart in terms of generating a higher percentage of optimized programs.

This fully validates our initial hypothesis that longer output sequences are harder to learn and lower the overall performance of the model. However, the diff-based representation comes at a cost of stricter well-formedness rules for applying the diff-based output. During the breakdown of all generated programs of SUPERSONIC in RQ1, we indeed showed that a portion of diff-based output representations are malformed (2045 out of 5590). While being potentially valid optimizations, this results in fewer programs being submitted for evaluation. Overall, the advantages of the sort diff representation win over the cons, and SUPERSONIC with diff-based output representation clearly outperforms the full program output representation.

RQ4: SUPERSONIC with the original diff-based output representation doubles the performance, compared to traditional full program output representation. While the majority of works in the field of code generation with deep learning use full program or full function output, our results clearly call for more exploration of shorter, contextual output representations such as SUPERSONIC's.

VI. DISCUSSION

A. Threats to Validity

1) Internal Threats:

a) Data leakage: The evaluation of GPT-3.5-Turbo and GPT-4 poses an internal threat in the form of potential data leakage that is beyond our control. The data used to train GPT-3.5-Turbo and GPT-4 is not publicly accessible, which means we lack visibility into the specific information it was exposed to during its training process. Consequently, there exists a potential risk of data leakage that GPT-3.5-Turbo and GPT-4 has already seen examples in the test set during its training process.

b) Functionally incorrect programs: We have noticed that some original programs are functionally correct according to the original metadata, but they failed when we submitted them ourselves. For Codeforces, the main reason was that they were either because of time/memory limit, or compilation error. For AIZU, it is a mix of compile error, time/memory limit, and wrong answer. We hypothesize that it is because of the compiler version being updated throughout the year. For these programs, we still count them as failures when calculating the %OPT value to ensure a fair comparison.

c) Measurement uncertainty: We also found that if we submit the same program twice to the Codeforces platform, we

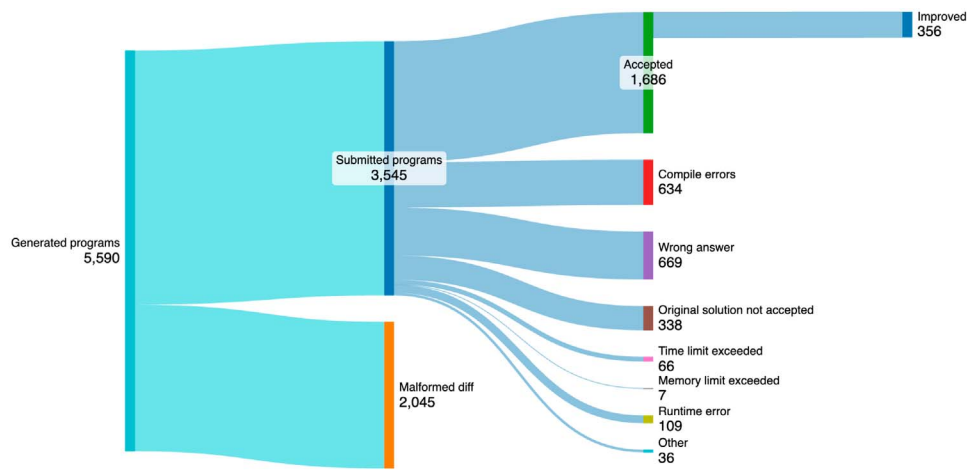


Fig. 5. Analysis of all programs generated by SUPERSONIC.

may get different execution times, e.g. 0ms and 15ms. This is because the Codeforces platform splits the execution time as different blocks, which means 0ms and 15ms are equal. Therefore when comparing execution time for programs on Codeforces, we divided all time into blocks of 16, i.e. 0 and 15 would be block 1, 16 and 31 would be block 2. Then we use the block number to calculate the speed up. Additionally, to account for noise, we also define a program to be more optimized if the time or memory improvement is at least 1.2, as mentioned in subsection IV-C.

d) Performance discrepancies with different prompts: One intriguing observation in our results is the performance discrepancy between GPT-3.5-turbo and GPT-4 when using advanced prompts. While GPT-4's performance improves with advanced prompts, GPT-3.5-turbo's performance declines. This discrepancy might be raised by several potential reasons: 1) The differences in architecture and training methodologies between GPT-3.5-turbo and GPT-4; 2) The effectiveness of prompt engineering might vary significantly between different model versions, one optimized prompt for one model being suboptimal for another model; 3) The datasets and evaluation metrics used in our study might also influence these results. Addressing these open questions is beyond the scope of this paper, and an important area for future research. Investigating when and why advanced prompts enhance performance in some models but not others would provide valuable insights into both prompt engineering and model behavior, leading to more effective utilization of language models.

2) External Threats: A significant external threat is that CodeNet and Codeforces datasets are both from code competition websites. They do not fully represent the diversity of real-world optimization problems. The problems on competition websites are designed with code optimizations in mind, and they facilitate the comparison between submissions by having functional correctness, running time, and memory consumption as part of their result. However, real-world optimizations are likely more varied. Future work may address this threat by creating new benchmarks for code optimization.

B. SUPERSONIC Limitations

Fig. 5 presents the evaluation of all programs generated by SUPERSONIC. The figure reveals that about 35% of the generated programs are malformed diffs that cannot be applied to the original program. For the remaining programs, approximately half could get accepted after submission to the corresponding code competition platforms. Approximately 20% (356/1686) of all accepted programs are able to strictly improve upon the original program. An interesting observation is that some accepted programs are trade-offs between reducing running time and increasing memory usage, or vice versa. Although these are not considered positive optimizations in our study, they could be beneficial in certain scenarios. As for the other half of the incorrect programs, compilation errors and incorrect answers to functional tests are the two primary issues. For the sake of open science, all suggested programs, including incorrect ones, are available for future researchers in our replication package.

C. Limitation of Baselines

In this paper, we consider zero-shot prompting for using LLMs. While we provide insights into the reasons for excluding other techniques such as few-shot learning in the preceding sections, we acknowledge the potential benefits of few-shot learning in enhancing the capabilities of LLMs, particularly in generating diverse outputs. For instance, by providing diff-based examples to an LLM like GPT-3.5-turbo, it might learn to generate outputs in a diff-based format [42]. This diffing capability might lead to enhanced performance on tasks such as code optimization, which is interesting for future research. Also, we did not fine-tune other LLMs with our approach such as Code LLaMA [43] or Mistral [44] because they are decoder models, where SUPERSONIC is a encoder-decoder model. Decoder models can be fine-tuned in a different way using parameter-efficient fine-tuning [45], [46]. It is a promising future research direction to adapt our work on decoder LLMs and to use parameter-efficient fine-tuning for automated code optimization with LLMs.

D. Code Optimization Versus Code Synthesis

Our extensive experience looking at predicted optimization has convinced us that the boundary between code optimization and code synthesis is hard to define. Consider a hypothetical example: an original program implements a naive algorithm to sort a list, such as bubble sort. In some cases, the distinction between optimization and synthesis is obvious, for example, if the model proposes to replace the original program with a quicksort or mergesort implementation. In other cases, if the model made many changes while retaining the core concept of bubble sort, the distinction is less clear.

The distinction between code optimization and full re-implementation has received limited attention in related research. We believe that clarifying this distinction is crucial in understanding the true capabilities of neural models. In this paper, we contribute to this clarification in a fully operational way. We draw the line between optimization and re-implementation by looking at the string similarity between the original and the predicted optimized program. This is a well-formed definition: a string similarity value of 0 is pure code synthesis, and a value close to 1 is a small change to the program. Nonetheless, we acknowledge that some optimizations might require rewriting the whole program, but this is considered out of scope in this paper.

E. Importance of Input/Output Code Representation

Designing the input and output representations of code for machine learning models is a critical aspect that demands careful consideration. The effectiveness of these representations can significantly impact the performance and efficiency of the resulting models.

Creating an appropriate input representation for code is akin to providing the model with the right set of tools for understanding and processing the task at hand. When designing input representations, it is essential to strike a balance between comprehensiveness and conciseness. The representation must capture the relevant information that enables the model to perform the desired task effectively. For the code optimization task, it is uncommon that we know in advance the source code location where we can optimize the program. In comparison with other code-related tasks such as the program repair task, we often have fault localization techniques that guide us toward the buggy locations. Therefore for SUPERSONIC, we decided to use the full program as input.

Crafting a concise and precise output representation for code is akin to presenting the model's insights and solutions in a simple manner, ensuring that the generated code conveys the intended functionality without unnecessary redundancy. Redundancy in the output is repeating information that was present in the input and increases the probability of unnecessary mistakes. Such mistakes are more unforgiving than mistakes in natural languages, as programs are interpreted by compilers with strict parser rules. In SUPERSONIC, we use the diff-based output representation to minimize the redundancy and risk of mistakes and only keep two context lines to localize the change. RQ3 proves that this design choice alone improves the performance of the model by 2 times. It shows the importance of designing a proper input and output representation for machine learning for code models that are rarely discussed.

F. Large Versus Small Models

It is widely accepted that larger models perform better, they are more sample efficient [12], are few-shot learners [47], and the performance increases with model size [48]. Furthermore, the accuracy and the robustness of a model also increase with dataset size [49]. One model that we compared against, GPT-3.5-Turbo, has at least 175B parameters and is trained on 500 billion tokens [12]. It demonstrated capabilities for many software engineering-related tasks, such as code generation [3], program repair [4], and code summarization [2]. In RQ2 of our study, we have also shown the good performance of GPT-3.5-Turbo on the code optimization task. However in RQ1, SUPERSONIC was able to outperform GPT-3.5-Turbo on the task of generating similar but optimized programs, despite being approximately 600x smaller (175B versus 278M) and trained with much less data. It shows the relevance of training a smaller model focusing on a single task with well-curated input and output representation.

G. Practical Usage of SUPERSONIC

SUPERSONIC can be used for code optimization in general, beyond code competitions. First, our general method for training a customized optimization LLM on a user-specific dataset can be tailored to any particular development context or company dataset. Second, once a SUPERSONIC model is trained, we envision equipping developers with a coding assistant that suggests optimizations on the fly in the IDE. This integration, which is pure engineering, is outside of the scientific scope of our paper but would be useful for companies deploying a SUPERSONIC -like technique.

VII. RELATED WORK

A. ML for Source Code Optimization

PIE4Perf is the most related work [6]. In this work, Madaan, Shypula, Alon, et al. investigates the ability of large language models to suggest performance-improving code edits. First, they extract samples of performance-improving code edits from the Codenet dataset. Then, they fine-tune the CodeGen model and they prompt Codex using few-shot prompting. They find that the system can generate performance-improving edits with speedups of more than 2.5x for over 25% of the programs, with a model 10x smaller than Codex to match its performance. The main differences between SUPERSONIC and PIE4Perf are: 1) we study both the running time and memory optimization while PIE4Perf only looks at running time optimization. 2) SUPERSONIC is based on an original diff-based output representation, while PIE4Perf outputs simple vanilla programs. 3) we take care of removing re-implementations in the evaluation procedure, yielding more meaningful measurements. 4) we report competitive results w.r.t. GPT-3.5-Turbo and GPT-4 with a model that is 600x and 3700x smaller.

DeepDev-PERF is a deep learning-based approach to improve software performance for C# applications [7]. The model is pre-trained on both English and source code corpora and fine-tuned on the task of performance improvement for C# applications. The evaluation shows that the model can generate the same performance improvement suggestions as the developer patches in 53% of the cases. The authors also submitted 19 pull requests with performance optimizations, of which 11 of these are accepted. SUPERSONIC is different from DeepDev-PERF by looking at C/C++ optimizations. The evaluation is also done by submitting to code competition websites that report the running time and memory consumption, instead of using the benchmark.

Chen, Tarlow, Swersky, et al. propose a discrete variational auto-encoder to extract discrete latent variables, each representing a code-edit that increases program performance [50]. The learned discrete latent variables can be used to guide programmers toward writing more efficient source code. They show that the discrete variational auto-encoder extracts better code efficiency edits than the Transformer baseline. SUPERSONIC on the other hand does not extract code efficiency edits but directly predicts the more optimized program in an end-to-end way.

RAPGen is an approach based on the OpenAI Codex model to do zero-shot code inefficiency fixing in C# methods [51]. This is done by first collecting a dataset of performance bug fixes by keyword matching on the commit message. The dataset is used to extract identifiers that are changed in the commits to form a knowledge base. The knowledge base is used to form an instruction to describe which identifiers were added/removed/edited when giving the input method. To predict the code inefficiency fix, they build a prompt by using the buggy method as a comment, followed by the instruction of which variables to change, and ending with the buggy method's signature. The prompt is used as input to the Codex model, and it generates the fixed method. They found that it performs better than DeepDev-PERF on exact match and CodeBLEU score, without any training.

Artemis++ is a tool for syntax-based optimization of C++ using a genetic algorithm [52]. The tool automatically chooses implementations of common data structures to provide performance improvements. It uses a genetic algorithm that automatically performs source code transformations and produces an optimized version of the program. The tool is evaluated on three C++ libraries, observing improvements by up to 16.09%, 27.90%, and 2.74% for CPU usage, runtime, and memory, respectively. SUPERSONIC is different in that it is trained to do more than data structure optimization, and relies on language model and seq2seq learning instead of a genetic algorithm.

LoopLearner is a tool that predicts the speedup of loop transformations [53]. They encode the source code as a vector and concatenate a compact encoding of transformations to the vector. The concatenated vector is then fed into CNN or RNN to predict the speedup of transformations. They found that by applying the top transformation, the program yields an average speedup of 1.29x. While LoopLearner only focuses on loops, SUPERSONIC 's input the whole program and can target different source code locations.

B. ML for Compiler Optimization

Ashouri et al. have a survey about compiler autotuning [54]. They summarize papers on two problems in the machine learning for compiler optimization field, 1) selecting the best optimization and 2) the phase-ordering of optimizations. Wang et al. focus more on the different machine-learning techniques used in the compiler optimization field.

Cummins et al. propose ProGraML, a graph-based program representation, that can be used as input to machine learning models [56]. The representation is a union of a control flow graph, data flow graph, and call flow graph. They show that on the task of choosing a CPU or GPU to run an OpenCL kernel, it surpassed prior approaches on all metrics.

DeepTune is a tool that predicts optimization heuristics on OpenCL kernel [57]. It directly uses the source code, instead of the binary code, as the input to an LSTM model to predict heterogeneous mapping (CPU or GPU) and OpenCL thread coarsening factor. The results show that by learning directly on source code, instead of manual features extracted from the binary code, DeepTune could match or surpass 89% of the predictions.

Narayanamurthy et al. use genetic algorithms to find the best set of compiler optimizations that optimized the performance while providing better error resilience [58]. The fitness function used in the genetic algorithm measures the error resilience of a candidate optimization. The resulting program is able to achieve similar performance as -O1, -O2, and -O3 while having better error resilience.

Schulte et al. use a genetic algorithm to reduce the non-functional properties of executables such as power efficiency [59]. The fitness function used in the genetic algorithm is hardware performance counters combined into a single scalar value. They find that they can reduce on average 20% energy usage on AMD and Intel systems.

Wang et al. use machine learning to partition stream processing programs [60]. The partition refers to dividing the program graph into clusters that are allocated to threads. They define different program features and used PCA to reduce the dimensionality to 10. Then, they use a k-means clustering algorithm to find the previous similar good partitions. They find that it achieves a 1.90x speedup over the already-tuned partition by the compiler.

Churchill et al. propose an approach to optimize loops in the Google native Client [61]. The approach has a bounded verifier that verifies the correctness of a loop transformation up to bound k. If it fails, it can create a counter-example that guides the search away from this transformation. Then they apply a sound verifier that uses strong loop invariants. In the evaluation, they achieve an average 25% speedup compared to libraries shipped by Google, and the optimized program can be formally verified.

Stoke is a tool that formulates the binary superoptimization problem as a stochastic search problem [62]. The search is guided by a cost function with a correctness term and a performance term. They use Markov chain Monte Carlo to sample the binary modification. By starting with a binary that is compiled

with -O0, Stoke was able to either match or outperform code produced by -O3.

Bunel et al. improve upon Stoke by replacing the searched distribution [63]. Stoke uses a uniform distribution to sample modifications of the program, and Bunel et al. instead used reinforcement learning to learn the distribution from past behaviors and program semantics. By doing so, they can increase the probability of reaching the best-optimized programs and generate better programs in fewer iterations.

Rotem et al. tackle the problem of improving the LLVM's BranchProbabilityInfo (BPI) heuristics [64]. They first use the profile-guided optimization (PGO) workflow to compile many different programs and record their branch probabilities. These data are used to train a model of gradient-boosted trees. Once the model is trained, it can be used to predict branch probabilities on programs without PGO workflow. They find that the geometric mean speedup of the new BPI heuristic across 10 workloads is 1.016.

Baghdadi et al. develop a deep learning-based cost model for predicting speedups of code transformations in the Tiramisu compiler [65]. The training data is created by generating random programs, applying a series of code transformations, and recording the actual speedups. The deep learning model is a combination of recurrent and recursive neural networks. They find that the proposed model has only 16% of mean absolute percentage error in predicting speedups on full programs and that it is able to find transformations that match or are better than SOTA compilers.

Agakov et al. use machine learning to speed up the process of iterative optimization on loops [66], i.e., iterative try different compilation optimization to determine the best set of compilation optimization. They extract different features from the loop and use PCA to represent the loop with a 5-dimension vector. This vector is used to determine the nearest neighbor in the training data and extract the best compilation optimization. They found that the average speedup is 1.22x on the T1 C6713 and 1.27x on the AMD Au1500.

Cummins et al. train a 7B-parameter transformer model from scratch to predict optimal compiler options and the resulting instruction counts before and after optimization, alongside generating the optimized code [67]. It marks a significant advancement by achieving a 3.0% reduction in instruction count over traditional compiler methods.

To recapitulate, none of the works presented in this section perform source code optimizations as SUPERSONIC does.

C. Other Work on Optimization

Petke et al. have a comprehensive survey about papers using genetic algorithms to improve programs [68]. They find that genetic algorithm has been able to improve the performance of a program for a diverse set of properties, such as execution time, energy, and memory consumption.

Marco et al. propose an approach to estimate the memory behavior of Spark applications to allow higher server utilization and system throughput [69]. They extract 22 raw features from the application and use PCA to reduce it to 5 principal components. To predict the function that best describes the memory usage, they use K nearest neighbor to find the most similar cluster. The evaluation shows that they achieve a 1.28x improvement on system throughput and 1.68x on turnaround time.

Chen et al. introduce a framework AutoTVM to optimize tensor operators that are used in deep learning models [70]. They use simulated annealing with a statistical cost function trained from past historical data to find more optimized low-level code. Experiments on existing frameworks show that it yields 1.2x to 3.8x performance improvement.

Ahn et al. propose Chameleon that improves upon AutoTVM by using reinforcement learning to optimize tensor operators [71]. Instead of using simulated annealing that relies on random walks, they use adaptive exploration by leveraging reinforcement learning. The result is that they achieve a 4.45x speed up in optimization time over AutoTVM.

VIII. CONCLUSION

In this paper, we have proposed SUPERSONIC, a tool that can generate source code optimizations for C/C++ programs with targeted changes. SUPERSONIC features an original diff-based output representation that is similar to a software patch. Our experiments clearly show that SUPERSONIC outperforms GPT-3.5-Turbo and GPT-4 on the task of source code optimization. We believe that there is a large research avenue in the area of using small but specific models targeting a given software engineering task instead of using larger general models.

REFERENCES

[1] C. Sadowski, J. Van Gogh, C. Jaspan, E. Soderberg, and C. Winter, "Tricorder: Building a program analysis ecosystem," in *Proc. IEEE/ACM 37th IEEE Int. Conf. Softw. Eng.*, vol. 1, Piscataway, NJ, USA: IEEE Press, 2015, pp. 598–608.

[2] H. Tian et al., "Is ChatGPT the ultimate programming assistant–how far is it?" 2023, *arXiv:2304.11938*.

[3] C. Liu et al., "Improving ChatGPT prompt for code generation," 2023, *arXiv:2305.08360*.

[4] D. Sobania, M. Briesch, C. Hanna, and J. Petke, "An analysis of the automatic bug fixing performance of ChatGPT," 2023, *arXiv:2301.08653*.

[5] R. A. Poldrack, T. Lu, and G. Beguš, "AI-assisted coding: Experiments with GPT-4," 2023, *arXiv:2304.13187*.

[6] A. Madaan et al., "Learning performance-improving code edits," 2023, *arXiv:2302.07867*.

[7] S. Garg, R. Z. Moghaddam, C. B. Clement, N. Sundaresan, and C. Wu, "DeepDev-PERF: A deep learning-based approach for improving software performance," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 948–958.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.

[9] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.

[10] A. Radford et al., "Improving language understanding by generative pre-training," *OpenAI*, 2018.

[11] M. Lewis et al., "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," 2019, *arXiv:1910.13461*.

[12] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.

[13] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 5485–5551, 2020.

[14] W. X. Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.

[15] Y. Chang et al., "A survey on evaluation of large language models," *ACM Trans. Intell. Syst. Technol.*, vol. 15, no. 3, pp. 1–45, 2024.

[16] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," 2020, *arXiv:2002.08155*.

[17] X. Gu, H. Zhang, and S. Kim, "Deep code search," in *Proc. 40th Int. Conf. Softw. Eng.*, 2018, pp. 933–944.

[18] X. Hu, G. Li, X. Xia, D. Lo, and Z. Jin, "Deep code comment generation," in *Proc. 26th Conf. Program Comprehension*, 2018, pp. 200–210.

[19] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," 2021, *arXiv:2109.00859*.

[20] S. Fang, T. Zhang, Y. Tan, H. Jiang, X. Xia, and X. Sun, "RepresentThemAll: A universal learning representation of bug reports," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 602–614.

[21] "Introducing chatGPT." OpenAI. Accessed: Nov. 30, 2022. [Online]. Available: https://openai.com/blog/chatgpt

[22] S. Fang, T. Zhang, Y. Tan, H. Jiang, X. Xia, and X. Sun, "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[23] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 27730–27744.

[24] N. M. S. Surameery and M. Y. Shakor, "Use chat GPT to solve programming bugs," *Int. J. Inf. Technol. & Comput. Eng. (IJITC)*, vol. 3, no. 1, pp. 17–22, 2023.

[25] W. Jiao, W. Wang, J.-t. Huang, X. Wang, and Z. Tu, "Is chatGPT a good translator? A preliminary study," 2023, *arXiv:2301.08745*.

[26] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," 2023, *arXiv:2304.03442*.

[27] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 27, 2014, pp. 3104–3112.

[28] C. Anil et al., "Exploring length generalization in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 38546–38556.

[29] K. Cho, B. Van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder-decoder approaches," 2014, *arXiv:1409.1259*.

[30] S. Zhou, U. Alon, S. Agarwal, and G. Neubig, "CodeBERTScore: Evaluating code generation with pretrained models of code," 2023, *arXiv:2302.05527*.

[31] A. Sagheer and M. Kotb, "Unsupervised pre-training of a deep LSTM-based stacked autoencoder for multivariate time series forecasting problems," *Sci. Rep.*, vol. 9, no. 1, 2019, Art. no. 19038.

[32] Y. Wu et al., "Google's neural machine translation system: Bridging the gap between human and machine translation," 2016, *arXiv:1609.08144*.

[33] R. Puri et al., "CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks," 2021, *arXiv:2105.12655*.

[34] J. W. Ratcliff et al., "Pattern matching: The gestalt approach," *Dr. Dobb's J.*, vol. 13, no. 7, 1988, Art. no. 46.

[35] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.

[36] A. Alali, H. Kagdi, and J. I. Maletic, "What's a typical commit? A characterization of open source software repositories," in *Proc. 16th IEEE Int. Conf. Program Comprehension*, Piscataway, NJ, USA: IEEE Press, 2008, pp. 182–191.

[37] R. Purushothaman and D. E. Perry, "Toward understanding the rhetoric of small source code changes," *IEEE Trans. Softw. Eng.*, vol. 31, no. 6, pp. 511–526, Jun. 2005.

[38] P. Weißgerber, D. Neu, and S. Diehl, "Small patches get in!" in *Proc. Int. Work. Conf. Mining Softw. Repositories*, 2008, pp. 67–76.

[39] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[40] S. Arora et al., "Ask me anything: A simple strategy for prompting language models," 2022, *arXiv:2210.02441*.

[41] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 22199–22213.

[42] P. Deligiannis, A. Lal, N. Mehrotra, and A. Rastogi, "Fixing rust compilation errors using LLMs," 2023, *arXiv:2308.05177*.

[43] B. Roziere et al., "Code Llama: Open foundation models for code," 2023, *arXiv:2308.12950*.

[44] A. Q. Jiang et al., "Mistral 7B," 2023, *arXiv:2310.06825*.

[45] N. Ding et al., "Parameter-efficient fine-tuning of large-scale pre-trained language models," *Nature Mach. Intell.*, vol. 5, no. 3, pp. 220–235, 2023.

[46] A. Silva, S. Fang, and M. Monperrus, "RepairLLaMA: Efficient representations and fine-tuned adapters for program repair," 2023, *arXiv:2312.15698*.

[47] J. Kaplan et al., "Scaling laws for neural language models," 2020, *arXiv:2001.08361*.

[48] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," 2022, *arXiv:2204.02311*.

[49] S. Lei, H. Zhang, K. Wang, and Z. Su, "How training data affect the accuracy and robustness of neural networks for image classification," 2019.

[50] B. Chen et al., "Learning to improve code efficiency," 2022, *arXiv:2208.05297*.

[51] S. Garg, R. Z. Moghaddam, and N. Sundaresan, "RAPGen: An approach for fixing code inefficiencies in zero-shot," 2023, *arXiv:2306.17077*.

[52] R. Giavrimis, A. Butler, C. C. Petrescu, M. Basios, and S. K. Dash, "Genetic optimisation of C++ applications," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE),* Piscataway, NJ, USA: IEEE Press, 2021, pp. 1180–1182.

[53] R. Mammadli, M. Selakovic, F. Wolf, and M. Pradel, "Learning to make compiler optimizations more effective," 2021, *arXiv:2102.13514*.

[54] A. H. Ashouri, W. Killian, J. Cavazos, G. Palermo, and C. Silvano, "A survey on compiler autotuning using machine learning," *ACM Comput. Surv.,* vol. 51, no. 5, pp. 1–42, 2018.

[55] Z. Wang and M. O'Boyle, "Machine learning in compiler optimization," *Proc. IEEE*, vol. 106, no. 11, pp. 1879–1901, Nov. 2018.

[56] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefler, and H. Leather, "ProGraML: Graph-based deep learning for program optimization and analysis," 2020, *arXiv:2003.10536*.

[57] C. Cummins, P. Petoumenos, Z. Wang, and H. Leather, "End-to-end deep learning of optimization heuristics," in *Proc. 26th Int. Conf. Parallel Architectures Compilation Techn. (PACT),* Piscataway, NJ, USA: IEEE Press, 2017, pp. 219–232.

[58] N. Narayanamurthy, K. Pattabiraman, and M. Ripeanu, "Finding resilience-friendly compiler optimizations using meta-heuristic search techniques," in *2016 12th Eur. Dependable Comput. Conf. (EDCC)*. IEEE, 2016, pp. 1–12.

[59] E. Schulte, J. Dorn, S. Harding, S. Forrest, and W. Weimer, "Post-compiler software optimization for reducing energy," *ACM SIGARCH Comput. Archit. News*, vol. 42, no. 1, pp. 639–652, 2014.

[60] Z. Wang and M. F. O'boyle, "Using machine learning to partition streaming programs," *ACM Trans. Archit. Code Optim.*, vol. 10, no. 3, pp. 1–25, 2013.

[61] B. Churchill, R. Sharma, J. Bastien, and A. Aiken, "Sound loop superoptimization for google native client," *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 313–326, 2017.

[62] E. Schkufza, R. Sharma, and A. Aiken, "Stochastic superoptimization," *ACM SIGARCH Comput. Archit. News*, vol. 41, no. 1, pp. 305–316, 2013.

[63] R. Bunel, A. Desmaison, M. P. Kumar, P. H. Torr, and P. Kohli, "Learning to superoptimize programs," 2016, *arXiv:1611.01787*.

[64] N. Rotem and C. Cummins, "Profile guided optimization without profiles: A machine learning approach," 2021, *arXiv:2112.14679*.

[65] R. Baghdadi, M. Merouani, M.-H. Leghettas, K. Abdous, T. Arbaoui, K. Benatchba et al., "A deep learning based cost model for automatic code optimization," *Proc. Mach. Learn. Syst.*, vol. 3, 2021.

[66] F. Agakov et al., "Using machine learning to focus iterative optimization," in *Proc. Int. Symp. Code Gener. Optim. (CGO'06),* Piscataway, NJ, USA: IEEE Press, 2006, pp. 11–pp.

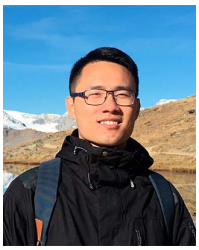
[67] C. Cummins et al., "Large language models for compiler optimization," 2023, *arXiv:2309.07062*.

[68] J. Petke, S. O. Haraldsson, M. Harman, W. B. Langdon, D. R. White, and J. R. Woodward, "Genetic improvement of software: A comprehensive survey," *IEEE Trans. Evol. Comput.*, vol. 22, no. 3, pp. 415–432, Jun. 2018.

[69] V. S. Marco, B. Taylor, B. Porter, and Z. Wang, "Improving spark application throughput via memory aware task co-location: A mixture of experts approach," in *Proc. 18th ACM/IFIP/USENIX Middleware Conf.*, 2017, pp. 95–108.

[70] T. Chen et al., "Learning to optimize tensor programs," 2018, *arXiv:1805.08166*.

[71] B. H. Ahn, P. Pilligundla, A. Yazdanbakhsh, and H. Esmaeilzadeh, "Chameleon: Adaptive code optimization for expedited deep neural network compilation," 2020, *arXiv:2001.08743*.



Zimin Chen received the M.S. and Ph.D. degrees in computer science from the KTH Royal Institute of Technology. He is currently working as a Research Engineer with Deutsche Telekom for the AI4Coding team. His research interest include intersection between machine learning and software engineering. Throughout the year, he has been involved in developing AI for software engineering tools used by large companies. Some of the projects he has worked on include automatic Java build/test repair, source and machine code optimization, mutation testing, and legacy code migration. His research interests include intersection between machine learning and software engineering.

Sen Fang received the master's degree in electronics and communication engineering from the Central China Normal University, in 2020. Currently, he is working toward the Ph.D. degree with the NC State University, in 2024 Fall. His research interests include intersection of software engineering and machine learning, particularly LLMs for code.

Martin Monperrus received the master's degree from Compiègne University of Technology and the Ph.D. degree from the University of Rennes. He is a Professor of software technology with KTH Royal Institute of Technology. His research interests include software engineering with a current focus on automatic program repair, program hardening, and chaos engineering.